M-B · First LLVM pass

6 tasks · B0 to B5

Six tasks. B0 closes three loose ends carried forward from M-A; they are cheap now and expensive later, so do them before anything else. B1 to B4 are LLVM mechanics and contain nothing CapsLock-specific. B5 exists because A2 established how the toolchain actually consumes LLVM: not through the submodule at all, but through an external llvm-config path with continuous-integration LLVM downloads disabled. Proving that handoff now, with a stock compiler and no CapsLock content, removes the largest remaining unknown from M-E at a cost of about a day. Expect the build system to consume more time than the code.

GATE B

The pass compiles, links, runs, and sees every memory access. M-A carry-forward closed, and the external-LLVM handoff proven with a stock toolchain.

HOW TO USE THIS FILE

Fill in the ANSWER box under each task and return this `.tex` file. Leave everything else untouched so the briefs stay comparable. Tick one status per task. Where the question asks for numbers, output or commit hashes, paste them into the `verbatim` block rather than describing them in prose. If a task is blocked, say what by; a blocked task and a slow task need different responses.

Compile with `pdflatex`. Return the source, not the PDF.

Name	Zephyr
Date submitted	2026.8.26
Toolchain commit	https://github.com/X-Bulow/capslock-artefacts
Branch / worktree	m-b-first-llvm-pass

B0 Carry-forward from M-A

OBJECTIVE

Close three loose ends from M-A before starting compiler work.

WHY THIS MATTERS

Two are measurement defects that will corrupt the G1 comparison if left in place, and one is an unverified complexity claim that both G1 and G2 depend on. All three cost hours now and weeks later.

METHOD

1. Rerun the A3 measurement at ten times the iteration count and confirm the ratios hold. The original absolute times were small enough that fixed startup cost may dominate. Note that the reference allocates a node pool of 65536 times 256 entries, roughly 1.7 GB of zero-initialised memory being faulted in, which is a startup cost rather than a per-access one.
2. Add a fifth datapoint: native x86-64 built without optimisation. The original step (b) mea-

sured stock emulation on an unoptimised binary, which conflates cross-ISA emulation cost with unoptimised-code cost. This separates them. That conflation was a defect in the brief, not in your work.

3. Add a test that counts nodes visited per access and demonstrates the count is bounded by tree depth plus nodes destroyed, rather than by tree size.

WATCH OUT FOR

— The third item matters most. Passing correctness tests is entirely compatible with a naive full-tree scan. G1 and G2 both assume amortised constant work per access; if the walk is linear in tree size that assumption fails silently and surfaces months later as disappointing performance with no obvious cause.

DONE WHEN

A revised ratio table with five variants, and a node-visit bound demonstrated by test.

YOU WILL BE ASKED

Do the A3 ratios hold at ten times the iterations, what is the native unoptimised number, and what is the maximum nodes-visited count your test observes?

Status: `[ ]` not started `[ ]` in progress `[x]` done `[ ]` blocked

ANSWER — B0

All three items complete.

The table reports five-trial arithmetic means of the internal `elapsed_ns` timer. A3 used 100 iterations; B0 used 1000 iterations on the same workload under AC power. Ratios are normalised to the unoptimised native x86-64 binary in each experiment.

Variant	A3 mean 100 iterations (s)	B0 mean 1000 iterations (s)	A3 ratio	B0 ratio
native x86-64, optimised	–	0.004490	–	0.051
native x86-64, unoptimised	0.014996	0.087388	1.000	1.000
stock QEMU, unoptimised RISC-V	0.170234	1.166708	11.352	13.351
CapsLock QEMU, checking no-op	1.456172	8.797251	97.104	100.668
full CapsLock QEMU	32.230098	181.941791	2149.245	2081.990

The A3 *native* executable was already the unoptimised `target/debug` binary (`opt-level=0`). The missing fifth point was therefore the optimised `target/release` native binary (`opt-level=3`), which is 19.464× faster than unoptimised native in B0. For the four configurations shared with A3, the ordering and main ratios hold: full/no-op changes from 22.133× to 20.682×, and full/unoptimised-native from 2149.245× to 2081.990×. All 25 B0 observations produced the same checksum.

Startup was already controlled in A3. As shown in `a3-b0-bench/src/main.rs`, `Instant::now()` is called only after input construction, compression and one warm-up decompression, and the timer ends immediately after the decompression loop. Thus `elapsed_ns` includes QEMU and capability work performed while executing the workload, but excludes Docker/QEMU process startup, ELF loading and shutdown. Using the same timing boundary in A3 and B0 prevents fixed startup from distorting the comparison and helps explain why their ratios remain broadly similar.

For item 3, the runtime now records capability nodes visited, nodes destroyed, and auxiliary interval-index probes for every access. Increasing the number of irrelevant siblings from 8 to 512 left the capability-node count unchanged at 2; index probes rose only from 3 to 9. A depth-64 chain visited 65 nodes, exactly the 65-node active path. An access that destroyed a 33-node conflicting subtree visited 35 nodes, exactly 2 active-path nodes plus the 33 nodes destroyed. The maximum observed count was therefore **65**. These cases demonstrate that capability-node visits

are bounded by active tree depth plus nodes actually destroyed, rather than by total tree size.

```
wide tree (8 siblings):    visited=2  destroyed=0  index-probes=3
wide tree (512 siblings):  visited=2  destroyed=0  index-probes=9
deep tree (depth=64):      visited=65 destroyed=0  index-probes=64
destroyed subtree:         visited=35 destroyed=33 index-probes=2
maximum capability nodes visited: 65
node-visit bound: PASS
```

Evidence

```
A3 summary: results/a3-timing-summary.txt
B0 raw:     results/b0-a3-1000-fiveway-ac-raw.txt
B0 summary: results/b0-timing-summary.txt
Timer:      a3-b0-bench/src/main.rs
Node bound: results/b0-nodecount-test.txt
```

B1 Build and install LLVM at the pinned revision

OBJECTIVE

Build LLVM at the exact revision the artefact pins, install it to a prefix, and verify the installed tree is consumable as an external LLVM.

WHY THIS MATTERS

A2 established that the artefact does not use the LLVM submodule at all. It patches a pinned upstream tree, installs that tree to a prefix, and points the Rust build at it through an llvm-config path. What you need is therefore an installed tree, not a build tree, and the two are genuinely different things. The same finding raises a second question: whether the artefact's LLVM patch is needed for your design at all.

METHOD

1. Build at the revision you recorded in A1, not at an arbitrary checkout of the same major version.
2. Configure with Ninja and the following: CMAKE_BUILD_TYPE set to Release; LLVM_ENABLE_ASSERTI on; LLVM_TARGETS_TO_BUILD limited to RISCV and X86; LLVM_ENABLE_PROJECTS set to include clang; LLVM_CCACHE_BUILD on; and CMAKE_INSTALL_PREFIX set to an explicit prefix of your choosing.
3. Run the build, then run the install target. The install step is the one that is easy to skip.
4. Verify by running llvm-config from the install prefix rather than from the build directory, querying version, prefix and libdir. Every path it reports should sit underneath your prefix.
5. Decide whether to apply the artefact's LLVM patch. Record the decision and the reasoning.

WATCH OUT FOR

— The two trees are different things. A build tree is what the build system produces in place: object files, libraries and tools that work only from where they sit, with headers still in your source checkout and some generated headers existing only under the build directory. An installed tree is the self-contained bin, include, lib and share layout that the install target copies to the prefix. llvm-config is a query tool that reports where headers, libraries and CMake files live, and it gives

different answers depending on which tree you ask. Asked from a build tree, some of those paths point back into your source checkout, so a consumer following them sees only part of what it needs. That fails confusingly rather than cleanly, which is worse.

— Include clang in the enabled projects. The Rust build needs it, and it is easy to omit when you are thinking only about opt and your own pass.

— Your out-of-tree plugin should consume the same installed tree. The CMake package lives under lib/cmake/llvm inside the prefix, so point LLVM_DIR there when you reach B2. That keeps the plugin and the Rust build consuming one LLVM rather than two, which is the whole reason for insisting on the install step.

— Your design most likely does not need the artefact's LLVM patch. Its contribution is custom instruction definitions and frame lowering, and you replace frame lowering with tagged allocas at C5 and emit no custom instructions at all. If that reasoning holds, build stock and the toolchain becomes materially simpler.

— Assertions are not optional. Without them, malformed IR fails silently or crashes far from its cause.

DONE WHEN

llvm-config run from the install prefix reports the expected version, and every path it reports sits under the prefix.

YOU WILL BE ASKED

What does llvm-config from your install prefix report for version, prefix and libdir, did you apply the artefact's LLVM patch, and why?

Status: [ ] not started [ ] in progress [x] done [ ] blocked

ANSWER — B1

I built and installed stock LLVM at the pinned revision `5399a24c66cb6164cf32280e7d300488c90d5765`.
The installed `llvm-config` reports LLVM 18.1.4, prefix `/home/bulow/projects/b1-llvm/install`, and libdir `/home/bulow/projects/b1-llvm/install/lib`. Its bin, include, lib and CMake-package paths are all beneath that prefix. The build is Release with assertions enabled, includes Clang, and is limited to the RISCV and X86 targets.
I did not apply the artefact's LLVM patch. That patch adds custom instruction definitions and changes frame lowering, whereas this design emits no custom instructions and replaces the frame-lowering change with tagged allocas at C5. Keeping LLVM stock therefore removes an unnecessary fork while allowing the B2 plugin and the B5 Rust build to consume the same installed LLVM tree.

Evidence — paste numbers, output or hashes below.

```
source commit:  5399a24c66cb6164cf32280e7d300488c90d5765
source status:  clean (stock LLVM; no artefact patch applied)
version:        18.1.4
prefix:         /home/bulow/projects/b1-llvm/install
bindir:         /home/bulow/projects/b1-llvm/install/bin
includedir:     /home/bulow/projects/b1-llvm/install/include
libdir:         /home/bulow/projects/b1-llvm/install/lib
cmakedir:       /home/bulow/projects/b1-llvm/install/lib/cmake/llvm
targets-built:  RISCV X86
```

```
assertion-mode: ON
full output:    results/b1-llvm-install.txt
```

B2 Hello-world plugin pass

OBJECTIVE

Build an out-of-tree pass that prints every function name.

WHY THIS MATTERS

Out-of-tree means seconds per rebuild instead of hours. You may never need to go in-tree; a proper driver flag is cosmetic and can wait until the thesis is written.

METHOD

1. Implement llvmGetPassPluginInfo and register a PassBuilder callback.
2. Build as a shared library and load it with the pass-plugin flag.

WATCH OUT FOR

— Most tutorials online use the legacy pass manager, with RegisterPass and runOnFunction overrides. Those are pre-2021 and will not work. You want the new pass manager.

— LLVM uses opaque pointers now, so getPointerElementType() no longer exists. Tutorials calling it are stale, and since this project is entirely about pointers you will hit this immediately. The error messages do not explain why.

DONE WHEN

It prints function names for a test .ll file.

YOU WILL BE ASKED

Paste the output on a two-function C file. Are you using llvmGetPassPluginInfo rather than RegisterPass?

Status: `[ ]` not started `[ ]` in progress `[x]` done `[ ]` blocked

ANSWER — B2

I built `CapsLockPass.so` as an out-of-tree shared-library plugin against the LLVM 18.1.4 installation from B1. The plugin is loaded by `opt` with `-load-pass-plugin`, and its function-pass pipeline name is `capslock-hello`. On IR generated from the two-function C test, it printed both defined function names.
The implementation uses the new pass manager: it exports `llvmGetPassPluginInfo` and registers `PB.registerPipelineParsingCallback`. It does not use the legacy `RegisterPass` or `runOnFunction` interfaces. Because this B2 pass only observes the IR, it returns `PreservedAnalyses::all()`.

Evidence — paste numbers, output or hashes below.

```
plugin: llvm-pass/build/CapsLockPass.so
command: opt -load-pass-plugin=CapsLockPass.so
```

```
             -passes='function(capslock-hello)' -disable-output

capslock-function: helper
capslock-function: main

registration: llvmGetPassPluginInfo
callback:     PB.registerPipelineParsingCallback
RegisterPass: absent
full output:  results/b2-hello-pass.txt
```

B3 Insert a runtime call

OBJECTIVE

Insert a call to your M-A runtime at each function entry.

WHY THIS MATTERS

This closes the loop from compiler to runtime for the first time. Everything afterwards is refinement of this one capability.

METHOD

1. Use IRBuilder and getOrInsertFunction to declare and call the runtime function.
2. Link the runtime into the test program.
3. Verify with a before-and-after IR dump.

WATCH OUT FOR

— Set up your debugging toolkit now: IR dumps before and after, the print-after-all flag, a DEBUG_TYPE with the debug-only flag, and llvm-reduce for shrinking failing cases. llvm-reduce in particular turns a crash on a 40,000-line crate into a ten-line reproducer, and most people do not know it exists.

DONE WHEN

A compiled program prints one line per function entered.

YOU WILL BE ASKED

Does the instrumented binary print one line per call at runtime? Show the IR diff for one function.

Status: `[ ]` not started `[ ]` in progress `[x]` done `[ ]` blocked

ANSWER — B3

I added `capslock_function_entry(const char *)` to the M-A runtime and implemented the `capslock-function-entry` module pass. The pass uses `getOrInsertFunction` to declare the hook and `IRBuilder` to insert one call at the first valid insertion point of every defined function. It skips declarations and the hook itself.
I compiled the instrumented IR together with `runtime/libcapslock.c` and ran the resulting

native x86-64 binary. The test enters `main` once and `helper` once, and the runtime printed exactly one line for each entry. The program exited successfully. The existing runtime suite also remained green: 9/9 unit tests passed, all 9 buggy oracle cases were flagged, all 9 clean variants passed, and the node-visit bound passed.
For debugging I verified IR diffs, `-verify-each`, `-print-after-all`, `DEBUG_TYPE` with `-debug-only=capslock-function-entry`, and the installed `llvm-reduce` 18.1.4.

Evidence — paste numbers, output or hashes below.

```
Runtime output:
  capslock-enter: main
  capslock-enter: helper
  exit_status=0

IR diff for helper:
  define dso_local i32 @helper(i32 noundef %value) #0 {
  entry:
+   call void @capslock_function_entry(ptr @0)
    %value.addr = alloca i32, align 4

Added declaration:
  declare void @capslock_function_entry(ptr)

IR diff:       results/b3-function-entry-ir.diff
runtime run:   results/b3-function-entry-runtime.txt
runtime tests: results/b3-runtime-regression.txt
debug-only:    results/b3-debug-only.txt
IR dump:       results/b3-print-after-all.txt
llvm-reduce:   results/b3-llvm-reduce.txt
```

B4 Enumerate memory accesses

OBJECTIVE

Find and characterise every memory access in the module.

WHY THIS MATTERS

These are the sites where revoke-on-use checks will go. Missing a category means silent false negatives later.

METHOD

1. Iterate instructions and handle LoadInst and StoreInst.
2. Get access size from the data layout, using the store size of the accessed type.
3. Log address and size.

WATCH OUT FOR

— Loads and stores are not all of it. Atomic read-modify-write, atomic compare-exchange, and the memcpy, memmove and memset intrinsics also touch memory. Note which you are not yet handling; you will need them by M-C.

DONE WHEN

Sizes are correct for i8, i32, i64 and struct-field access.

YOU WILL BE ASKED

Are sizes correct for i8, i64 and a struct field load? Which memory-accessing instructions are you not yet handling?

Status: `[ ]` not started `[ ]` in progress `[x]` done `[ ]` blocked

ANSWER — B4

`capslock-memory-access` walks every instruction and handles `LoadInst`/`StoreInst`: accessed type from `getType()`/`getValueOperand()->getType()`, size from `DataLayout::getTypeStoreSize()`. Struct-field size is correct because the type comes from the `getelementptr`-narrowed pointer, not the struct itself.
Not-yet-handled categories are matched with `isa<>` and logged explicitly rather than skipped silently: `AtomicRMWInst`, `AtomicCmpXchgInst`, `MemCpyInst`/`MemMoveInst`/`MemSetInst`, their element-wise atomic counterparts (`AtomicMemCpyInst` etc. — a separate class hierarchy, not caught by the non-atomic `isa<>` checks), `VAArgInst`, and the `llvm.masked.{load,store,gather,scatter}` intrinsics. Plain atomic loads/stores are already covered, since they use ordinary `LoadInst`/`StoreInst`.
Verified sizes: 1 byte (`i8`), 8 bytes (`i64`), 8 bytes for a struct's `long` field (not the struct's 16-byte total). Each unhandled category produced exactly one log line and no size. Reran `capslock-function-entry` on `two-functions.ll`: output unchanged from the committed B3 result, so no regression.

Evidence — paste numbers, output or hashes below.

```
results/b4-memory-access.txt    (llvm-pass/test/memory-access.c)
results/b4-extra-categories.txt (llvm-pass/test/extra-categories.ll)

store i8, size=1 bytes / store i64, size=8 bytes / store i64, size=8 bytes (p->b)
atomicrmw / cmpxchg / llvm.memcpy: not yet handled
va_arg / llvm.memcpy.element.unordered.atomic: not yet handled
```

B5 Prove the external-LLVM handoff

OBJECTIVE

Build a stock Rust compiler against your own LLVM installation, with no CapsLock content whatsoever.

WHY THIS MATTERS

A2 identified the exact seam by which the toolchain consumes LLVM, and that seam is the largest remaining unknown in M-E. Testing it now with a stock compiler costs about a day. Discovering it is fragile at E1, four milestones later and with your own modifications in the mix, costs considerably more and is much harder to diagnose.

METHOD

1. Take a stock Rust checkout at the revision you recorded in A1.

2. Generate config.toml with continuous-integration LLVM downloads disabled and llvm-config pointing at your install tree, following the mechanism you documented in A2.
3. Build, and confirm the resulting compiler reports your LLVM version.

WATCH OUT FOR

— No CapsLock content in this task at all. If it fails, the failure is in the handoff, which is precisely what you are testing.

— If the handoff proves fragile or needs changes, say so explicitly. The scope of M-E depends on the answer.

DONE WHEN

A stock Rust compiler built against your LLVM reports your LLVM version.

YOU WILL BE ASKED

Does the compiler report your LLVM version when queried verbosely, and did anything about the external-LLVM handoff need changing?

Status: `[ ]` not started `[ ]` in progress `[x]` done `[ ]` blocked

ANSWER — B5

Yes. A stock checkout at the A1-recorded revision, built with `download-ci-llvm = false` and `llvm-config` pointed at the B1 install (native `x86_64-unknown-linux-gnu`, no RISC-V cross-linker needed to test the handoff itself), produces a stage-2 `rustc` that reports `LLVM version: 18.1.4` and the exact A1 commit hash — no CapsLock content anywhere in the checkout or the config.

One thing needed changing: bootstrap's sanity check for an externally configured LLVM requires a `FileCheck` binary in `llvm-config`'s `bin/` directory. B1's install was built with `LLVM_INSTALL_UTILS=OFF`, so `FileCheck` existed in the LLVM build tree but was never installed. Fixed by copying the already-built binary into the install tree; no LLVM reconfiguration or rebuild required. Otherwise the handoff was not fragile — the two `config.toml` keys A2 documented were sufficient.

Evidence — paste numbers, output or hashes below.

```
$ build/host/stage2/bin/rustc -vV
rustc 1.78.0-nightly (c69fda7dc 2024-03-11)
commit-hash: c69fda7dc664e62f8920a02a4e55d6207b212c24
commit-date: 2024-03-11
host: x86_64-unknown-linux-gnu
release: 1.78.0-nightly
LLVM version: 18.1.4

Fix required: cp <b1-llvm-build>/bin/FileCheck <b1-llvm-install>/bin/FileCheck
```

GATE B -- SIGN-OFF

The pass compiles, links, runs, and sees every memory access. M-A carry-forward closed, and the external-LLVM handoff proven with a stock toolchain.

`[x]` Gate met `[ ]` Not met

If not met, state which task is outstanding and why:

```
N/A -- B0 through B5 all done with evidence; see individual answerboxes.
```